

Job-Ready dbt + Databricks E-Commerce Project [2026]

End-to-end data pipeline for an e-commerce analytics layer: **orders**, **customers**, **products**, and **order line items**. Built with dbt on Databricks so you can run it locally or deploy via CI/CD.

What you get

Medallion architecture (no prefix/postfix on schema names):

- **Bronze** (**bronze** schema): Raw data from seed CSVs (customers, orders, order_items, products) loaded into Databricks.
- **Silver** (**silver** schema): Staging views for orders, order_items, products (1:1 with raw) + intermediate views. Customers have no staging—only **scd_customers** (snapshot from raw) in gold.
- **Gold** (**gold** schema): Marts — **fct_orders**, **fct_order_items**, **dim_products**; **scd_customers** (Type 2 SCD — the only customer table, attributes only, no order-derived fields); and **exposures** (who uses the data).

Concepts included (beginner-friendly):

- **Incremental models:** **fct_orders** and **fct_order_items** use **merge** so only new/changed data is processed each run.
- **Slowly changing dimension (Type 2):** **scd_customers** snapshot tracks customer history; use **dbt_valid_from** / **dbt_valid_to** for point-in-time queries.
- **Exposures:** Link gold models to dashboards and consumers (CEO dashboard, product analytics).
- **Hooks:** **on-run-start** / **on-run-end** in **dbt_project.yml** for run logging.
- **Tests:** Generic (unique, not_null, relationships, accepted_values) plus singular tests (revenue non-negative, no future dates, positive quantities, valid email, positive product price).

Catalog: **workspace**. Workspace: **dbc-aef35066-afa2.cloud.databricks.com**.

Prerequisites

- **Python 3.12** (recommended; 3.10–3.11 also work; 3.14 is not yet supported by dbt's dependencies)
- A Databricks workspace with a **SQL Warehouse** (or all-purpose cluster)
- Databricks **host**, **HTTP path**, and **token** (or OAuth) for connection

Quick start

1. Clone and install

```
git clone <your-repo-url>
cd job-ready-dbt-databricks-data-engineering-project
python3.12 -m pip install -r requirements.txt
```

2. Connect to Databricks

Option A — project profile (recommended): Create `profiles.yml` in the project (or copy from `profiles.yml.example`) and set:

- `http_path`: from Databricks → SQL Warehouse → Connection details → HTTP path
- `token`: from User Settings → Developer → Access tokens

Then run with:

```
export DATABRICKS_HTTP_PATH="/sql/1.0/warehouses/xxxxx"
export DATABRICKS_TOKEN="dapi..."
DBT_PROFILES_DIR=. dbt seed && dbt run && dbt test
```

Option B — user profile: Copy `profiles.yml.example` to `~/.dbt/profiles.yml` and fill in `http_path` and `token`. Catalog is `workspace`; schema names are `bronze`, `silver`, `gold` (no prefix/postfix).

3. Install packages, load seeds, run models and snapshots

```
dbt deps
dbt seed
dbt snapshot # scd_customers from raw (must run before run, as models depend
on it)
dbt run
dbt test
```

4. Build and test (full run)

```
# If using project profile (profiles.yml in repo with env vars):
export DATABRICKS_HTTP_PATH="/sql/1.0/warehouses/YOUR_ID"
export DATABRICKS_TOKEN="dapi..."
DBT_PROFILES_DIR=. dbt seed
DBT_PROFILES_DIR=. dbt run
DBT_PROFILES_DIR=. dbt snapshot
DBT_PROFILES_DIR=. dbt test
```

Or use the helper script (uses Python 3.12, runs seed + run + test):

```
export DATABRICKS_HTTP_PATH="/sql/1.0/warehouses/YOUR_ID"
export DATABRICKS_TOKEN="dapi..."
./run.sh
```

5. Docs (optional)

```
dbt docs generate
dbt docs serve
```

Project layout (medallion)

```
seeds/          # raw_*.csv → bronze schema
models/
  staging/      # silver: stg_* (views)
  intermediate/ # silver: int_* (views)
  marts/core/   # gold: fct_orders, fct_order_items (incremental),
dim_products, exposures
snapshots/      # gold: scd_customers (Type 2 SCD – only customer table,
attributes only)
tests/          # singular tests (assert_*.sql) + schema tests in yml
macros/         # generate_schema_name (schema as-is, no prefix/postfix)
```

CI/CD (GitHub Actions)

Deploy bundle (DAB) — dev and prod

Two workflows deploy the **Databricks Asset Bundle** by branch:

Branch	Workflow	Target	GitHub environment
dev	<code>.github/workflows/deploy-bundle-dev.yml</code>	dev	dev
main	<code>.github/workflows/deploy-bundle.yml</code>	prod	prod

- **Push to dev** → validate and deploy to **dev** (`databricks bundle deploy -t dev`). Job and files go to the dev bundle path in the workspace.
- **Push to main** → validate and deploy to **prod** (`databricks bundle deploy -t prod`).

Required GitHub secrets (per environment or repo): `DATABRICKS_HOST`, `DATABRICKS_TOKEN`. Configure in **Settings** → **Secrets and variables** → **Actions** (repo-level) or per **Environment** (dev / prod) if you use different workspaces or tokens.

Local deploy:

```
export DATABRICKS_HOST="https://your-workspace.cloud.databricks.com"
export DATABRICKS_TOKEN="dapi..."
databricks bundle deploy -t dev    # or -t prod
```

Override `warehouse_id`, `catalog`, or `dbt_schema` in `databricks.yml` or via `--var` if needed.

dbt CI workflow (optional)

If you add a workflow (e.g. [.github/workflows/dbt.yml](#)) that runs dbt on PRs, use these secrets as needed:

Secret	Description
<code>DATABRICKS_HTTP_PATH</code>	SQL Warehouse HTTP path
<code>DBT_SCHEMA</code>	Schema for CI runs
<code>DATABRICKS_CATALOG</code>	Unity Catalog name

Interview talking points

When asked "Walk me through a data project you've built" or "What's in your dbt project?", you can say:

- **Architecture:** "I built a medallion pipeline on Databricks: bronze for raw data, silver for staging and intermediate models, gold for fact and dimension tables. Schema names are clean—no prefix or postfix."
- **Facts & dimensions:** "Gold has two fact tables—`fct_orders` (order grain) and `fct_order_items` (line grain)—and one dimension, `dim_products`. Customers are in `scd_customers` only (Type 2 SCD, attributes only, no order aggregates)."
- **Scale & history:** "The fact tables are incremental with merge so we only process new data. Customer changes are tracked in `scd_customers` for point-in-time reporting."
- **Quality & impact:** "I added generic tests (unique, not_null, relationships, accepted_values) and singular tests for business rules. Exposures link gold models to dashboards so we can see downstream impact."
- **Deployment:** "The pipeline runs in CI on every push via GitHub Actions against Databricks, and can be scheduled with Databricks Jobs for production."

See [docs/GOLD_LAYER_QUESTIONS.md](#) for the dimensional model and 30+ business questions answerable from the gold layer.

Production tip: When raw data comes from a lake or warehouse instead of seeds, define **sources** in YAML and set **source freshness** so dbt can alert when data stops landing.

Databricks workflow (DAB + native dbt task)

The pipeline is defined as a **Databricks Asset Bundle** so job and resources deploy from the repo.

- `databricks.yml` — bundle root: bundle name, variables (`warehouse_id`, `catalog`, `dbt_schema`), and target `prod`.
- `resources/dbt_shopflow_job.yml` — job `dbtXdatabricks_shopflow` with one **dbt task** (serverless): `project_directory` = deployed bundle root, schema `bronze`, catalog `workspace`, commands `dbt deps`, `dbt seed`, `dbt snapshot`, `dbt run`, `dbt test`, and environment `dbt-default` (dbt-databricks ≥1.0, <2.0).

Push to dev or merge to main runs the corresponding deploy workflow, which syncs the repo and deploys/updates the job. Run the job from the Databricks Jobs UI or:

```
export DATABRICKS_HOST="https://your-workspace.cloud.databricks.com"
databricks bundle run dbtXdatabricks_shopflow -t dev # or -t prod
```

Legacy `databricks/job_dbt_pipeline.json` is kept for reference; the canonical definition is the bundle in `resources/`.

License

See [LICENSE](#).

#only for testing

```
(.venv) PS E:\Daily_Databricks\anirvan_databricks_dbt\job-ready-dbt-databricks-data-engineering-project-main\job-ready-dbt-databricks-data-engineering-project-main> git status
```

On branch master

No commits yet

Untracked files:

(use "git add ..." to include in what will be committed)

.github/

.gitignore

LICENSE

README.md

Use_Daily_Practice.txt

databricks.yml

dbt_project.yml

docs/

macros/

models/

package-lock.yml

packages.yml

profiles.yml.example

raw_customers.csv

requirements.txt

resources/

seeds/

snapshots/

tests/

nothing added to commit but untracked files present (use "git add" to track)

```
(.venv) PS E:\Daily_Databricks\anirvan_databricks_dbt\job-ready-dbt-databricks-data-engineering-project-main\job-ready-dbt-databricks-data-engineering-project-main> git push
```

fatal: The current branch master has no upstream branch.

To push the current branch and set the remote as upstream, use

```
git push --set-upstream origin master
```

To have this happen automatically for branches without a tracking upstream, see 'push.autoSetupRemote' in 'git help config'.

(.venv) PS E:\Daily_Databricks\anirvan_databricks_dbt\job-ready-dbt-databricks-data-engineering-project-main\job-ready-dbt-databricks-data-engineering-project-main>